Matrix Computation Notes

Bang-Shien Chen*

Abstract

Notes and assignments of Matrix Computation (Fall 2023) at National Taiwan Normal University by Prof. Tsung-Ming Huang. All results are available at https://github.com/doggydoggy0101/math_project, conducted on a desktop computer with an Intel i9-13900 CPU, 128GB RAM.

1 Laplacian

In this lecture, we focus on Laplacian matrices, which is obtained by some graph and its adjacency matrix. Let $\mathcal{G} = \big(\mathcal{V}(\mathcal{G}), \mathcal{E}(\mathcal{G})\big)$ be a connected simple graph of n vertices and m edges in which $\mathcal{V}(\mathcal{G}) = \{v_1, ..., v_n\}$ and $\mathcal{E}(\mathcal{G}) = \{e_{ij} | v_i \sim v_j\}$. The adjacency matrix A of $\mathcal{G}$ is an $n \times n$ matrix defined as:

$$A_{i,j} = \begin{cases} 1 \ (w_{ij} \text{ if weighted}) & \text{if } v_i \sim v_j \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

The Laplacian matrix L of $\mathcal{G}$ is an $n \times n$ matrix defined as:

$$L_{i,j} = \begin{cases} -1 \ (-w_{ij} \text{ if weighted}) & \text{if } v_i \sim v_j \\ \deg(v_i) \ (\sum_{k \neq i} w_{ik} \text{ if weighted}) & \text{if } i = j \\ 0 & \text{otherwise} \end{cases} \tag{2}$$

Here is an example of adjacency matrix and Laplacian matrix obtained by a graph text:

$$A\text{:} \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix}, \quad L\text{:} \begin{pmatrix} 3 & -1 & -1 & -1 \\ 1 & 2 & -1 & 0 \\ -1 & -1 & 3 & -1 \\ -1 & 0 & -1 & 2 \end{pmatrix}$$

`graph.txt`

```
1 2
2 3
3 4
4 1
1 3
```

0905 assignment Download graph data from the website "The KONECT Project" to generate a Laplacian matrix.

This is relatively simple, we do not present the results here while it can be generated by `laplacian.m`[3].

*ID: 61240018s
Email: dgbshien@gmail.com

2 Power Method

Power method is an iterative algorithm that produces the largest eigenvalue. The basic idea is as follows: Let $(\lambda_i, x_i),\ i = 1, ..., n$ be eigenpairs of matrix A. We assume that $|\lambda_1| > |\lambda_i|$ for i larger than 1, i.e., λ_1 is the largest eigenvalue if all eigenvalues are positive. Note that $\{x_1, ..., x_n\}$ is linear independent and $A^k x = \lambda^k x$. Given any vector $u \neq 0$ with $||u||_2 = 1$, exists $\alpha_1, ..., \alpha_n$ such that $u = \alpha_1 x_1 + ... + \alpha_n x_n$. We can decompose $A^k u$ as $\alpha_1 \lambda^k x_1 + ... + \alpha_n \lambda^k x_n$, that leads to:

$$\frac{1}{\lambda_1^k} A^k u = \alpha_1 x_1 + (\frac{\lambda_2}{\lambda_1})^k \alpha_2 x_2 + ... + (\frac{\lambda_n}{\lambda_1})^k \alpha_n x_n \to \alpha_1 x_1 \text{ as } k \to \infty \tag{3}$$

We can now calculate $Au_1, A^2 u_2, ...$ and so on to approximate (λ_1, x_1).

Algorithm 1 Power method

Input: A: matrix
Output: λ: eigenvalue; u: eigenvector
  u = random vector with norm 1
  while iteration and tolerance condition **do**
    $v = Au$
    $\lambda' = ||v||_2$ or $l(v)$
    $u = \frac{v}{\lambda'}$ ▷ update u
    $\lambda = \lambda'$ ▷ update λ
  end while

Note that we use the norm function $|| \cdot ||_2$ in order to maintain convergence, however, we could also use a linear function $l(v)$ instead. An example is to use the first element of v (`v[0]`), where it has lower computation time in compared with the norm function.

0912 assignment Use power method to compute the largest eigenvalue of L and the associated eigenvector. Please try the matrix size to be 10K, 100K, 1000K and use `tic` and `toc` to count the cpu time.

Results can be generated by `hw1_power_method.m`[3]. Table 1 shows a comparison between 3 datasets with matrix size larger than 10K, 100K, 1000K respectively.

Table 1: Power method with different matrix size.

Dataset	dim	iters	time(s)	$\|\|Ax - \lambda x\|\|_2$
brightkite	58228	114	0.065401	0.006493
gowalla	196591	43	0.136336	0.010979
Pennsylvania	1088092	626	6.393967	0.000707

3 Inverse Power Method

Inverse power method with fixed shift is also an iterative algorithm to find an eigenvalue closest to a given scalar σ. Let $(\lambda_i, x_i),\ i = 1, ..., n$ be eigenpairs of matrix A, $|\lambda_1 - \sigma| < |\lambda_i - \sigma|$ for i larger than 1, i.e., λ_1 is the eigenvalue closest to σ. Let $\mu_i = 1/(\lambda_i - \sigma)$ such that $|\mu_1| > |\mu_2| \geq ... \geq |\mu_n|$. In other words, we solve the matrix $(A - \sigma I)$ with eigenvalues μ_i by power method in order to generate the dominant eigenvalue μ_1, where $\lambda_1 = \mu_1^{-1} + \sigma$ is the eigenvalue of matrix A that is closest to σ.

Algorithm 2 Inverse Power method with fixed shift

Input: A: matrix; σ: scalar
Output: λ: eigenvalue; u: eigenvector
  $A_{\text{shift}} = A - \sigma I$
  $u =$ random vector with norm 1
  while iteration and tolerance condition **do**
    $v = A_{\text{shift}}^{-1} u$
    $\mu' = l(v)$
    $u = \frac{v}{\mu'}$
    $\lambda = 1/\mu' + \sigma$
  end while

The convergence of Algorithm 2 is linear, we next introduce inverse power method with variant shifts, where the convergence could result in quadratic or even cubic in symmetric case. The idea is to update σ as well as the shifted matrix during the procedure. By Newton's method, $(A - \lambda_k I)u_{k+1} = (\lambda_{k+1} - \lambda_k)u_k$. Let $v_{k+1} = u_{k+1}$, we get $(A - \lambda_k I)v_{k+1} = u_k$, which is similar to the previous shift inverse power method but instead of a fixed σ, we shift the matrix with λ_k at the $k + 1^{th}$ step.

Algorithm 3 Inverse Power method with variant shifts

Input: A: matrix; σ: scalar
Output: λ: eigenvalue; u: eigenvector
  $\lambda_k = \sigma$
  $u =$ random vector with norm 1
  while iteration and tolerance condition **do**
    $A_{\text{shift}} = A - \lambda_k I$
    $v = A_{\text{shift}}^{-1} u$
    $\mu' = l(v)$
    $u = \frac{v}{\mu'}$
    $\lambda_k = 1/\mu' + \lambda_k$
  end while

As mentioned in previous section, we use $l(v) = v(1)$ for an intuitive example. Setting $l(v) = v(\text{idx})$ in Algorithm 3 where $\text{idx} = \text{argmax}(u)$, i.e., the index of

the largest eigenvalue of the initial guess u, results in better stability. Next, recall the Rayleigh quotient is defined as:

$$R(A, x) = \frac{x^t A x}{x^T x} \tag{4}$$

If (λ, x) is an eigenpair of matrix A, $\frac{x^t A x}{x^T x} = \lambda \frac{x^t x}{x^T x} = \lambda$. Note that if we keep $||u|| = 1$ during the procedure, we could replace λ in Algorithm 3 as $u^T A u$.

Algorithm 4 Inverse Power method with Rayleigh quotient

Input: A: matrix; σ: scalar
Output: λ: eigenvalue; u: eigenvector
  $\lambda_k = \sigma$
  $u =$ random vector with norm 1
  while iteration and tolerance condition **do**
    $A_{\text{shift}} = A - \lambda_k I$
    $v = A_{\text{shift}}^{-1} u$
    $\mu' = ||v||_2$ ▷ keep $||u|| = 1$
    $u = \frac{v}{\mu'}$
    $\lambda_k = u^t A u$ ▷ replace λ
  end while

0919 assignment Use shift inverse power method to compute the eigenpair (λ, x) of L, where λ is the nearest shift value $\sigma = 0.01$. You can use `x = A\b` to solve linear system $Ax = b$. Please try to use different matrix size of L and count the cpu time for computing the target eigenpair.

Results can be generated by `hw2_inverse_power_method.m`[3]. We compare Algorithm 2 (SII), Algorithm 3 (NII) and Algorithm 4 (RQI) using a dataset from Table 1. We also test $\sigma = 100$ as and $\sigma = 1000$ as shown in Table 2.

Table 2: Inverse power methods comparison.

	brightkite (dim=58228)							
	$\sigma = 0.01$		$\sigma = 100$			$\sigma = 1000$		
Algo	ev	t(s)	eigval	iters	time(s)	eigval	iters	time(s)
SII	0	15.76	100.19	89	465.16	1056.02	18	93.11
NII	0	**6.42**	*99.49*	7	36.27	1056.02	5	25.93
RQI	0	15.54	100.19	6	**31.14**	1056.02	5	**25.78**
SII-LU	0	59.91	100.19	36	47.84	1056.02	15	47.31

RQI is the fastest in most cases, while one of NII case's λ_k shifted away and converged to another eigenvalue which is slightly further to given σ. Another observation is that we solve $v = A_{\text{shift}}^{-1} u$ by `x =A\b`, where this is a Cholesky solver since our Laplacian matrix is symmetric[4]. However, we could first apply LU factorization to a matrix and solve $v = U^{-1} L^{-1} u$ by `x =A\b` where this is

now a triangular solver because L, U are triangular matrices[4]. For SII, because the shifted matrix is fixed, we first apply LU factorization before the iteration. Therefore, we solve $v = U^{-1}L^{-1}u$ during the iteration, which could result in faster computation time. For large dimensional matrices, we can also apply permutation `p=amd(A)` to reorder the matrix and solve LU decomposition more robust.

1003 assignment Use Matlab function `amd` to find the approximate minimum degree permutation Π of $A - \sigma I$, where A is a Laplacian matrix of a graph and $\sigma \neq 0$ is a shift value. Let U_1 and U_2 be the LU factorization upper matrix of $A - \sigma I$ and $\Pi^T(A - \sigma I)\Pi$ respectively,

- Compare the CPU time of the LU factorization for $A - \sigma I, \Pi^T(A - \sigma I)\Pi$.
- Compare the sparsity of U_1 and U_2 (Use `spy(U)` to plot the sparsity pattern of the matrix).
- Apply the LU factorization of $\Pi^T(A - \sigma I)\Pi$ to solve the linear system $(A - \sigma I)x = b$ and count the associated CPU time.

Results can be generated by `hw3_permutation.m`[3]. We show the plot of the sparsity pattern of U_1 and U_2 in Figure 1, apparently, the upper matrix after applying permutation has a better structure. Note that solving linear system of larger dimension sparse matrices by LU factorization without permutation is unwise, for instance, it takes more than 5 hours to plot Figure 1 in our case, `x=A\b` is a much faster choice.

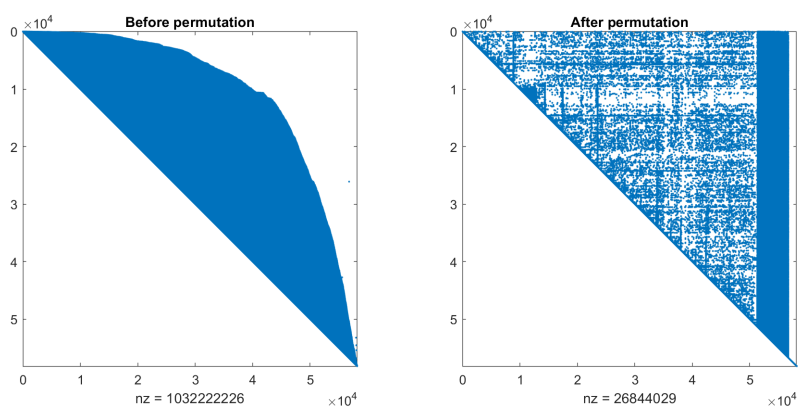

Figure 1: Applying permutation to the brightkite dataset in Table 1.

1010 assignment Compare the CPU time for computing the associated eigenpair by inverse power method and inverses power method with LU factorization.

The results of applying LU factorization on inverse power method with fixed shift (SII-LU) is also shown in Table 2 (`hw2_inverse_power_method.m`[3]). The larger dataset's computation time of the iteration part of $\sigma = 100$ and $\sigma = 1000$ is only 1.006 and 0.417 seconds, while most of the computation time is caused by the LU factorization part before the iteration with 47.835 and 46.894 seconds respectively. Note that the computation time of SII-LU is always around 47 seconds, we could define a threshold that if other iteration method with variant shifts cost more than 47 seconds, we choose to use SII-LU instead.

4 Krylov Method

It is shown in Table 2 that inverse power method does not always converge to the same point, this section gives a more stable method to find eigenvalues.

4.1 Arnoldi Method

Recall that power method calculates $Au, A^2u, ...$ to approximate the largest eigenvalue, now we consider the Krylov subspace defined as

$$\mathcal{K}_k = \text{span}\{u, Au, A^2u, \ldots, A^{k-1}u\}, \tag{5}$$

as well as the Krylov matrix defined as

$$K_k = \begin{bmatrix} u & Au & A^2u & \ldots & A^{k-1}u \end{bmatrix}. \tag{6}$$

Theorem 1 *Let $\mathcal{K}$ be an eigenspace of A, V be an orthonormal basis for $\mathcal{K}$, there exists an unique H such that $AV = VH$.*

By Theorem 1, let (λ, x) be an eigenpair of A, then $(\lambda, V^\dagger x)$ is an eigenpair of H. On the other hand, if (λ, s) is an eigenpair of H, then (λ, Vs) is an eigenpair of A. The goal here is to first find the Krylov subspace of A as well as its orthonormal basis V_k, then compute $H_k = V_k^\dagger A V_k$ and use the eigenpair of H_k to approximate the eigenpair of A. Note that we can use QR factorization to decompose $\mathcal{K}_k(A, u)$ into $V_k R_k$, where V_k is an orthonormal basis and R_k is an upper triangular matrix. Due to the fact that generating a Krylov matrix by Equation 6 is unstable, we next introduce an algebraic method to construct K.

Theorem 2 *Let $||u||_2 = 1$, $K_{k+1}(A, u)$ be full column rank matrix, $U_{k+1} = \begin{bmatrix} u_1 & \ldots & u_{k+1} \end{bmatrix}$ be the Q-factor of K_{k+1}, then there exists an unreduced upper Hessenberg $\hat{H}_k \in \mathbb{C}^{(k+1)\times k}$ such that $AU_k = U_{k+1}\hat{H}_k$. This is also known as the Arnoldi decomposition.*

Conversely, we have $U_{k+1}^\dagger U_{k+1} = I_{k+1}$ and $AU_k = U_{k+1}\hat{H}_k$, then U_{k+1} is the Q-factor of the QR decomposition of K_{k+1}. So given a normalized u_1, we can construct U_{k+1}, which is also the Q-factor, i.e., the orthogonal basis V. Now let's look closer at the unreduced upper Hessenberg $\hat{H}_k$:

$$\hat{H}_k = \begin{bmatrix} h_{11} & h_{12} & \cdots & h_{1k} \\ h_{21} & h_{22} & \cdots & h_{2k} \\ & \ddots & \ddots & \vdots \\ & & h_{k,k-1} & h_{k,k} \\ & & & h_{k,k+1} \end{bmatrix} \equiv \begin{bmatrix} H_k \\ h_{k+1,k} e_k^T \end{bmatrix}. \tag{7}$$

Set $\beta_k = h_{k+1,k}$, we have:

$$AU_k = U_{k+1}\hat{H}_k = U_k H_k + \underbrace{\beta_k u_{k+1} e_k^T}_{v}. \tag{8}$$

Consider only the kth column, we get $Au_k = U_k h_k + \beta_k u_{k+1}$. Applying left multiplication $U_k^\dagger$, we get $U_k^\dagger A u_k = U_k^\dagger \cdot U_k h_k + U_k^\dagger \cdot \beta_k u_{k+1} = h_k + 0$. Therefore, with $h_k = U_k^\dagger A u_k$, we can compute the normalized $\beta_k = ||Au_k - U_k h_k||_2 / ||u_{k+1}||_2 = ||Au_k - U_k h_k||_2$ by orthonormality and update $u_{k+1} = \beta_k^{-1}(Au_k - U_k h_k)$. To conclude, we first solve h_k in Equation 8 by orthonormality, then solve u_{k+1} again with orthonormality. Note that the algorithm could fail to produce orthogonal vectors, we can combine the first step with Gram-Schmidt algorithm:

Algorithm 5 Arnoldi method

Input: A: matrix
Output: U: orthogonal vectors; H: Krylov approximation of A

```
u_1 = random vector with norm 1
while iteration and tolerance condition do
    v = A u_k
    for i = 1, 2, ..., k do                    ▷ modified Gram-Schmidt
        h_{k,i} = u_i^† v
        v = v − h_{k,i} u_i
    end for
    h_{k+1,k} = ||v||_2
    u_{k+1} = v / h_{k+1,k}
end while
```

4.2 Lanczos Method

Now let's consider symmetric cases, $U_k^T A U_k = T_k$ is a symmetric tridiagonal:

$$T_k = \begin{bmatrix} \alpha_1 & \beta_1 & & & & \\ \beta_1 & \alpha_2 & \beta_2 & & & \\ & \beta_2 & \alpha_3 & \beta_3 & & \\ & & \ddots & \ddots & \ddots & \\ & & & \alpha_{k-2} & \alpha_{k-1} & \beta_{k-1} \\ & & & & \beta_{k-1} & \alpha \end{bmatrix}. \tag{9}$$

We can rewrite Equation 8:

$$AU_k = U_k T_k + \beta_k u_{k+1} e_k^T \tag{10}$$

This is also known as the Lanczos decomposition. Consider only the kth column, we get $Au_k = \alpha_k u_k + \beta_{k-1} u_{k-1} + \beta_k u_{k+1}$. Similar to the Arnoldi method, we first compute $\alpha_k = u_k^T A u_k, \beta_k = ||Au_k - \alpha_k u_k - \beta_{k-1} u_{k-1}||_2$ by orthonormality and than solve $u_{k+1} = \beta_k^{-1}(Au_k - \alpha_k u_k - \beta_{k-1} u_{k-1})$ again with orthonormality.

Algorithm 6 Lanczos method

Input: A: matrix
Output: U: orthogonal vectors; T: Krylov approximation of A

```
u_1 = random vector with norm 1
v = A u_1
α_1 = v^† u_1
β_1 = ||v − α_1 u_1||_2
u_2 = β^{-1}(v − α_k u_1)
while iteration and tolerance condition do          ▷ k ≥ 2
    v = A u_k
    α_k = v^† u_k
    β_k = ||v − α_k u_k − β_{k−1} u_{k−1}||_2
    u_{k+1} = β_k^{-1}(v − α_k u_k − β_{k−1} u_{k−1})
end while
```

Here we plot the 30 largest eigenvalues of the gowalla dataset from Table 1 by Algorithm 5 (Arnoldi) and Algorithm 6 (Lanczos), which could be generated by `krylov.m`[3]. The blue and red circles are the eigenvalues found by Arnoldi method and Lanczos method, where the error $||Ax - \lambda x||_2$ is 0.0 and 1563.4 respectively. If you try more times, you'll see that Lanczos method is unstable with larger error. However, there are a lot of variations for improvement, for instance, the implicitly restarted Lanczos method, we will not discuss here.

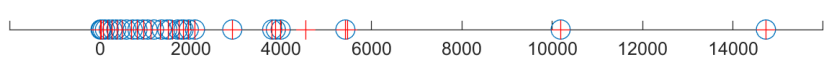

Figure 2: Eigenvalues of the gowalla dataset in Table 1.

1024 assignment Use Krylov method to compute the m smallest non-negative eigenvalues and the associated eigenvectors of the Laplacian matrix L with $m = 30$.

- Use Matlab function `eigs`.
- Use Gaussian elimination to solve linear system in `eigs`.
- Use LU factorization to solve linear system in `eigs`.

- Use iterative method to solve linear system in `eigs`.

This is simply specifying different functions in `eigs`, all the results can be generated by `hw4_krylov_method.m`[3].

References

[1] G.H. Golub and C.F. Van Loan, *Matrix Computations*, The Johns Hopkins University Press, (1983).

[2] R.A. Horn and C.R. Johnson, *Matrix Analysis*, Cambridge University Press, (1985).

[3] B.-S. Chen, https://github.com/doggydoggy0101/math_project.

[4] https://www.mathworks.com/help/matlab/ref/mldivide.html